

作业书写提示：同学可以直接编辑本文档，在选择题的括号中、简答题的题目之后，用醒目的颜色填写答案；也可以另写一份仅包含答案的文档，但需注意标明题号。提交的作业须为 PDF 格式。

Part 1. 进程与线程（50 分）

一、选择题（共 20 分，每题 2 分）

1. 一个进程映像是（ ）。
 - A. 由协处理器执行的一个程序
 - B. 一个独立的程序+数据集
 - C. PCB 结构与程序和数据的组合
 - D. 一个独立的程序
2. 进程与程序的根本区别是（ ）。
 - A. 静态和动态特点
 - B. 是不是被调入内存
 - C. 是不是具有就绪、运行和等待三种状态
 - D. 是不是占有处理器
3. 在任何时刻，一个进程的状态变化（ ）引起另一个进程的状态变化。
 - A. 必定
 - B. 一定不
 - C. 不一定
 - D. 以上都不是
4. 进程之间交换数据不能通过（ ）途径进行。
 - A. 管道
 - B. 消息传递系统
 - C. 访问目标进程的地址空间
 - D. 共享内存
5. （ ）必定伴随进程切换。
 - A. 一个进程创建后，进入就绪态
 - B. 一个进程从运行态变为就绪态
 - C. 一个进程从阻塞态变为就绪态
 - D. 以上答案都不对
6. 下列说法中，不正确的是（ ）。
 - A. 一个进程可以创建一个或多个线程
 - B. 一个线程可以创建一个或多个线程

- C. 一个线程可以创建一个或多个子进程
 - D. 一个进程可以创建一个或多个子进程
7. 【2014 统考真题】一个进程的读磁盘操作完成后，操作系统针对该进程必做的是（ ）。
- A. 修改进程状态为就绪态
 - B. 降低进程优先级
 - C. 给进程分配用户内存空间
 - D. 增加进程时间片大小
8. 【2014 统考真题】下列关于管道（pipe）通信的叙述中，正确的是（ ）。
- A. 一个管道可实现双向数据传输
 - B. 管道的容量仅受磁盘容量大小限制
 - C. 进程对管道进行读操作和写操作都可能被阻塞
 - D. 一个管道只能有一个读进程或一个写进程对其操作
9. 在多对一的线程模型中，当一个多线程进程中的某个线程被阻塞后，（ ）。
- A. 该进程的其他线程仍可继续运行。
 - B. 整个进程都将阻塞
 - C. 该阻塞线程将被撤销
 - D. 该阻塞线程将永远不可能再执行
10. 下面关于用户级线程和内核级线程的描述中，错误的是（ ）。
- A. 多处理器系统可以调度同一进程中的多个内核级线程并行运行。
 - B. 跨进程调度用户级线程不需要内核参与，控制简单
 - C. 大多数通用操作系统通常允许用户程序内部实现和调度用户级线程
 - D. 若系统不支持一个进程内有多个内核级线程，则系统的调度单位是进程

二、简答题（共 30 分）

1. 在采用优先级进程调度时，运行中的进程是否一定包括系统中优先级最高的进程？为什么？（4 分）
2. 某单处理机系统只考虑运行态、就绪态、阻塞态。进程 P 初始处于运行态，

随后依次发生以下事件：P 请求读取磁盘，但所需数据尚未准备好；磁盘读取完成；调度程序选中 P 占用 CPU；P 的时间片用完，但尚未执行结束。请按事件发生顺序直接写出进程 P 的完整状态变化序列，并说明每次状态转换的直接原因。

(10 分)

3. `fork()`进程创建与输出分析。假设 `fork()`不会失败，`printf()`是原子的。阅读以下 C 程序：

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int x = 1;
    pid_t pid;

    pid = fork();           // F1
    if (pid == 0) {
        x += 1;
        if (fork() == 0) {  // F2
            x += 1;
            printf("A: x=%d\n", x);
        } else {
            printf("B: x=%d\n", x);
        }
    } else {
        x += 10;
        printf("C: x=%d\n", x);
    }
    printf("D: x=%d\n", x);
    return 0;
}
```

(1) 直接写出程序一共创建了几个进程（包括初始进程）。然后，给出除初始进程 P0 之外的每个进程被哪个进程在哪一行 `fork()`调用中创建。约定用 P1、P2 等标号指代除 P0 之外的进程；用程序注释中的 F1、F2 指代 `fork()`调用。

(2 分)

(2) 直接写出程序的一种可能输出, 要求完整写出每一行输出。然后回答: 父子进程中对 `x` 的修改会互相影响吗? 为什么? (5 分)

(3) 假设在语句“`x += 10`”之前, 紧邻该语句, 插入语句“`wait(NULL);`”。“`wait(NULL);`”的作用是什么? 它对输出顺序有什么影响? 要求在回答中用你在 (1) 中的标号明确指出该语句所涉及的进程, 并直接写出输出中哪些行之间的顺序关系是可以确定的, 哪些是不能确定的, 可以确定的顺序是什么。(5 分)

(4) 用 `sleep()` 调整该程序中不同进程之间的输出关系, 使一个进程的输出在另一进程之后, 是绝对可靠的吗? 为什么? (`sleep()` 的作用是: 将调用它的进程设置为睡眠态, 持续指定时间后才可继续执行。假设睡眠中的进程不会被意外唤醒。) (4 分)

Part 2. 进程与线程（50 分）

一、选择题（共 14 分，每题 2 分）

1. 进程 P 正在执行临界区代码时因等待磁盘 I/O 而阻塞。对于与该临界资源对应的同一个临界区，下列说法正确的是（ ）。
 - A. 另一个进程可以立即进入该临界区，因为 P 已经不占用 CPU
 - B. 另一个进程不能进入该临界区，但可以被调度执行其临界区之外的代码
 - C. 任何进程都不能被调度，直到 P 的 I/O 完成
 - D. 另一个进程可以进入该临界区，但不能修改临界资源
2. 计数信号量 S 采用如下约定：S > 0 时 S 表示可立即获得的资源数，S = 0 表示无可用资源且无等待者，S < 0 时 -S 表示等待进程数。若当前 S = 1，设 M 为可立即获得的资源数、N 为阻塞等待的进程数，则（ ）。
 - A. M = 0, N = 1
 - B. M = 1, N = 0
 - C. M = 1, N = 2
 - D. M = 2, N = 0
3. 两个进程使用初值为 1 的互斥信号量 mutex 保护同一临界区。若 P(mutex) 不是原子操作，最直接的风险是（ ）。
 - A. 两个进程都可能读到 mutex = 1，并都将其改为 0 后进入临界区
 - B. mutex 一定会变成负数，因此两个进程都被阻塞
 - C. V(mutex) 无法把阻塞进程唤醒
 - D. 以上都不是
4. 有 4 个并发进程共享一段程序，每次最多允许 3 个进程同时进入。用计数信号量 S（初值为 3）实现限制，且 P(S) 在 S = 0 时使进程阻塞，则 S 在整个运行过程中可能取到的所有整数值是（ ）。
 - A. 4, 3, 2, 1, 0
 - B. 3, 2, 1, 0, -1
 - C. 2, 1, 0, -1, -2
 - D. 3, 2, 1, 0, -1, -2
5. 一个有界缓冲区有 N 个空槽位，使用信号量 empty = N、full = 0、mutex

= 1。生产者每次放入一个数据，且放入操作必须在 **mutex** 保护下完成。生产者放入一个数据时，以下操作顺序正确的是（ ）。

- A. **P(empty); P(mutex); 放入数据; V(mutex); V(full)**
- B. **P(mutex); P(empty); 放入数据; V(full); V(mutex)**
- C. **P(empty); 放入数据; P(mutex); V(mutex); V(full)**
- D. **P(full); P(mutex); 放入数据; V(mutex); V(empty)**

6. 在抽象并发模型中，共享整数 **count** 初值为 0。两个进程各执行一次 **count++**；每次 **count++** 明确分为“读取 **count**”和“将读取值加 1 后写回 **count**”两步，且没有任何同步。假设每次单独的读或写不可再分，则最终 **count** 可能为（ ）。

- A. 只能为 0
- B. 只能为 1
- C. 只能为 2
- D. 为 1 或 2

7. 下列选项中（ ）不是产生死锁的必要条件。

- A. 互斥使用资源
- B. 占有并等待
- C. 不可剥夺
- D. 进程按固定顺序轮流获得 CPU 时间片

二、简答题（共 36 分）

1. 一个生产者 **PA** 和一个消费者 **PC** 共享有 **N** 个槽位的有界缓冲区（**N > 0**）。**PA** 每次生产 1 个数据并放入缓冲区，**PC** 每次从缓冲区取出 1 个数据；放入和取出操作必须互斥，缓冲区不能溢出或下溢。请用信号量 **empty**、**full**、**mutex** 和 **P**、**V** 操作实现 **PA**、**PC** 对缓冲区的互斥访问。写出这些信号量在你的实现中的初值和含义，然后给出 **PA**、**PC** 循环写入或读取缓冲区的伪代码。

约定：信号量 **empty** 表示空槽数，**full** 表示已有数据数，**mutex** 用于保护缓冲区操作。用描述性名称、中文、注释等方式使伪代码易读。（9 分）

2. 理发店里有一位理发师、一把理发椅和 **n** 把供等候顾客坐的椅子。若没有顾客，理发师等待（阻塞）；有顾客到来时，理发师为他理发。若理发师正在理

发时又有顾客来到，若有空椅子可坐，则顾客坐下来等待（阻塞），否则离开（退出函数）。用信号量和 P、V 操作实现理发师循环服务顾客和单个顾客接受服务（或等待、离开）的行为，提供伪代码，并在伪代码之前说明所用信号量和辅助变量的定义和初值。

约定：在理发师要理发、顾客要接受理发时，理发师调用一次 `cut_hair()`，顾客调用一次 `get_haircut()`。认定有一方调用函数后理发开始，双方调用均返回时理发结束。如果 `get_haircut()` 被并发调用，认定程序出错。如果一次 `cut_hair()` 调用与多次 `get_haircut()` 调用在时间上重叠，认定程序出错；反之亦然。你在伪代码中定义的全局变量默认是在进程间共享的。

提示：在你的伪代码中，理发师的实现须形如：

```
barber() {
    while (true) {
        ...
        cut_hair();
        ...
    }
}
```

顾客的实现须形如：

```
customer() {
    ...
    get_haircut();
    ...
}
```

`barber()` 只被调用一次，永不返回；`customer()` 可被并发调用。（15 分）

3. 下表给出了整型信号量 S 的 `wait()` 和 `signal()` 操作的功能描述，以及采用开/关中断指令在单核机器上实现信号量操作互斥的两种方法。

功能描述	方法 1	方法 2
<code>Semaphore S;</code> <code>wait(S) {</code> <code>while (S <= 0);</code> <code>S = S - 1;</code> <code>}</code>	<code>Semaphore S;</code> <code>wait(S) {</code> 关中断; <code>while (S <= 0);</code> <code>S = S - 1;</code> 开中断; <code>}</code>	<code>Semaphore S;</code> <code>wait(S) {</code> 关中断; <code>while (S <= 0) {</code> 开中断; 关中断; } <code>S = S - 1;</code> <code>}</code>

<pre>signal(S) { S = S + 1 }</pre>	<pre>signal(S) { 关中断; S = S + 1 开中断; }</pre>	<pre> 开中断; } signal(S) { 关中断; S = S + 1 开中断; }</pre>
--	--	--

请回答下列问题：

- (1) 为什么在 `wait()` 和 `signal()` 操作中对信号量 `S` 的访问必须互斥执行？
(2 分)
- (2) 分别说明方法 1 和方法 2 是否正确，并说明理由。(6 分)
- (3) 用户程序能否使用开/关中断指令实现临界区互斥？为什么？(4 分)

Part 3. 附加题（10 分）

加分规则：附加题分数与第一次作业分数相加，即本次作业总分多 10 分，并计入课程四次作业总分，但四次作业总分不超过课程考核标准中规定的分数占比。

为助力国产操作系统生态建设，本课程组联合华为设置以下**选做任务**，同学们自愿参与，完成任务可获得平时作业额外加分。

任务：HarmonyOS 应用开发者基础认证

同学们完成学习并通过认证考试，提交合格证书即可申请加分，认证考试不超过 60 分钟。认证入口：

<https://developer.huawei.com/consumer/cn/training/dev-cert-detail/101666948302721398>

合格证书样例：

